Pre-registered analysis plan

Analysis plan fixed before the test data were examined; reproduced here with identifying details removed.

Anonymisation edits (and nothing else): the method's name is replaced by "the method"; person names by "the authors"; hostnames, links, file paths, repository and commit identifiers, artifact URLs and the name of a commercial LLM are removed or replaced by a generic description; clock times are kept without time zone. Content, structure, decision rules and the dated lock and deviation entries are unchanged.

Pre-registration: does the method detect ambiguous queries better than retrieval-only predictors?

Status: LOCKED (Stage 1), 2026-10-15 19:07, v5. Locked by the authors. v1 was drafted 2026-10-15 10:35; v2 (15:15) applies the red-team fixes; v3 (16:20) and v4 (16:50) apply the dry-run findings; v5 (18:35) registers Dense-QPP. The change lists are at the end. Mock results: kept privately by the authors (link removed).

Two-stage lock.

- **Stage 1 (design):** the version-controlled change that sets this line to `LOCKED`.
- **Stage 2 (code):** before any dev feature or dev label is computed, the hash of the frozen analysis script is appended under "Freeze record". The dev split is then evaluated exactly once, by a script whose only output is the tables below.

Template: the authors' internal methods-review document (section 6). Facts about AmbigQA are from Min et al. 2020, read 2026-10-15.

Question (claim 1 of 2; no LLM involved)

On open-domain questions with human ambiguity labels, does the method's score rank ambiguous questions above clear ones better than the best predictor that uses only the retrieval results?

Data

- **Questions:** AmbigQA / AmbigNQ (`sewon/ambig_qa`, CC BY-SA 3.0), full config.
  - **Fit:** `train` (10,036 questions; one generator per question, no validation; 47% ambiguous). Split 80/20 into fit and val by question id, seed 0.
  - **Test:** `dev` (2,002 questions; two generators plus a validator; 51% ambiguous). Test labels are not public.
- **Ambiguity label (primary):** 1 if at least one annotation is `multipleQAs`, else 0. Decided by the authors, 2026-10-15.
  - Train has a single annotation, so the label is noisier there than on dev. AUROC is insensitive to prevalence, but this mismatch is reported.
- **Retrieval label (secondary):** 1 if the top-1 chunk comes from `nq_doc_title`.
- **Corpus (label-independent by construction):**
  - The source is the AmbigQA Wikipedia dump of 2020-01-20, text CC BY-SA 3.0.
  - It holds the pages named in `nq_doc_title` over train and dev, and nothing else. `viewed_doc_titles` is **not used**: annotators open more pages for ambiguous questions, so including them leaks the label into the corpus.

  - Each page is cut into ~250-token chunks (the paper's C250), keeping at most the first 5 chunks per page. Hard cap: 50k chunks. If the cap is exceeded, train questions are subsampled (seed 0); the dropped questions' pages are removed **and those questions leave the fit set**, so no fitted question lacks its gold page. Dev pages are always kept.
  - Known remaining bias: every gold page is in the corpus, so retrieval is easier than in real use.
- **Embedders:** `all-MiniLM-L6-v2` (primary) and `all-mpnet-base-v2` (robustness), with HF revision hashes recorded at Stage 2. Retrieval uses FAISS `IndexFlatIP` on unit vectors, k = 50 (depth 1,000 for Dense-QPP's lists).

Methods

- **Code:** the method's reference library at a pinned, pushed commit (identifier withheld for anonymity), installed with the constraint `pyarrow<18`. All seeds are 0 unless swept.
- **The method (primary):** the library pipeline. That means topic clustering (UMAP/HDBSCAN, seeded), the 9 default features, `KernelDensity(bw=0.2)` on balanced classes, then `p_correct`. Ambiguity score = 1 − `p_correct`. It is fitted once per label, on the fit split.
- **Ablations:** topology only (`w1_h0`, `lt_max_h1`); the full set minus topology.
- **Retrieval-only predictors (the comparison set).** The best of these on **val** is "the baseline" for P1.
  - Library B1–B6: top-1 score, score gap, score entropy, mean kNN distance, kNN density, and Mahalanobis distance to the top-k centroid.
    - B4 and B5 are monotone transforms of the same mean distance, so they rank identically and are reported as one.
  - Query-performance-prediction (QPP) family:
    - NQC (dense adaptation): the std of the top-k cosine similarities ÷ their mean.
    - Coherence: the mean pairwise cosine among the top-k chunk embeddings.
    - Clarity (simplified, unweighted): the KL divergence between the unigram distribution of the top-k chunk text and the corpus distribution. Tokens are lowercase `\w+`, add-one smoothed.
    - **Dense-QPP (adaptation; included by the authors, 2026-10-15 18:30):** Arabzadeh et al., CIKM 2023, checked against the paper and its authors' public code release (last push 2023-06-26), both read 2026-10-15. Score = mean over 30 perturbations of the average overlap between the query's top-K list and the top-K list for $q + s \cdot z$.
      - z: 30 standard-normal vectors drawn once (seed 0) and shared by every question. The paper says "we have sub-sampled 30 noises" and "the same noise is used for all the queries"; the code uses one unseeded vector per noise level.
      - s is the noise **standard deviation**, from {0.01, …, 0.09}. The paper's text calls it a variance, but its code passes it as the std (`np.random.normal(0, var, 384)`).
      - No renormalisation after adding the noise, as in the code. With inner-product search, rescaling the query would not change its ranking anyway.
      - Similarity: average overlap, i.e. the `rbo` package's default `p = 1.0`, which the code uses. The paper says "Ranked Bias Overlap". Verified equal to a port of that package to 2e-15.
      - K from {100, 200, …, 1000}, the paper's grid. The (s, K) pair (90 variants) is tuned on the **fit** split, by |AUROC − 0.5|. Tuning on val would inflate Dense-QPP's val AUROC and could make it "the baseline" without being the strongest, which would favour the method in P1.
      - Higher overlap means a more robust query (easier, per the paper). Its sign for ambiguity is fixed on val, as for every unsupervised predictor.
  - **Direction spread:** R = ‖mean of the k unit offsets (chunk − query)/‖ · ‖‖. This is the simplest non-topological statistic of the same point cloud Rips sees. The mean pairwise cosine of the offsets equals $(kR^2 - 1)/(k - 1)$, so it is the same statistic and not added.
- **Sign rule:** each unsupervised predictor's direction (whether higher means ambiguous) is fixed on val, never on dev.
- **Also fitted on the same features:** B7 calibrated logistic and B8 GBM, both via the library's own classes. B9 oracle is used for the secondary label only.
- **KDE balancing** is seeded. The library's own balancing was unseeded, which is fixed in a pinned library commit; the analysis uses identical logic with a seed. The vectorised `p_correct` is asserted equal to the library scorer's correctness-probability method.
- **Controls:**

- Random rotation: features must match to 1e-9, otherwise the run is invalid as a bug.
- Random neighbourhoods: AUROC ≈ 0.5.
- **Isotropic null:** 2,000 sets of k i.i.d. uniform directions in the embedder's dimension.

Endpoints and decision rules (paired bootstrap, B = 10,000, on dev)

#	Comparison	"Win" if	"Not demonstrated" if
P1 (primary)	The method vs the baseline chosen on val	ΔAUROC ≥ 0.02 and the 95% CI excludes 0	CI includes 0, or $\Delta < 0.02$
S1	Full − (full minus topology)	$\Delta > 0.01$ and CI excludes 0	$\Delta \leq 0.01$ (internal methods review, §4.4)
S2	Topology-only vs direction spread R	$\Delta > 0.01$ and CI excludes 0	Otherwise: the topology adds nothing over the spread of directions
S3	The method's KDE vs B7 (same features)	$\Delta > 0.01$	Within 0.01: the KDE is not load-bearing
S4 (sanity floor)	Real `w1_h0` and `lt_max_h1` distributions vs the isotropic null	Two-sample KS $p < 0.01$ for both. **Necessary, not evidence:** almost any real neighbourhood passes	Otherwise: the features are indistinguishable from noise directions
S5	P1 repeated on the retrieval label	as P1	as P1
S6	Topology-only KDE vs the baseline chosen on val	as P1	as P1

- Choosing the baseline on val removes the max-of-many selection from dev, so P1 is a single comparison and needs no multiplicity correction. This replaces v1's Bonferroni over 5.
- **Power, stated before the run.** This is a Hanley–McNeil approximation with n = 2,002, about 51% positives, and an assumed correlation of 0.5 between the two scores. SE(ΔAUROC) ≈ 0.013 and the 95% CI half-width ≈ 0.025. P1's power is ≈ 0.9 at a true Δ of 0.04 and ≈ 0.35 at 0.02. **A "not demonstrated" outcome does not rule out a true Δ up to ≈ 0.025.**
- **Sensitivity:** k $\in \{25, 50, 100\}$, $\in \{\text{none}, 1.4\}$, bandwidth $\in \{0.2, \text{Scott}\}$. Seeds 0–4 are averaged within each cell. A cell whose seed-averaged P1 estimate has the opposite sign to the primary cell **and** $|\Delta| \geq 0.01$ is a flip, and any flip is reported as a non-result. Cells with $|\Delta| < 0.01$ are reported as ≈ 0.
- **arm:** the mean fraction of neighbours kept at $= 1.4$ is reported. If it is $\geq 95\%$, the arm is labelled vacuous, not counted as robustness.
- Also reported but not used to decide anything: AUPRC, Brier, ECE (10 equal-mass bins), per-embedder results, and latency per query for each method separately.

Registered predictions (written before any data)

1. AmbigQA's ambiguity is mostly about events (39%), properties (27%) and time (13%), with entity references at only 23% (Min et al. 2020, Table 1, n = 100). The method's geometry targets topical scatter, not dates inside one article. **We predict that the method's AUROC for time-type ambiguous vs clear questions is lower than for other ambiguous vs clear questions.**
   - Stratifier, frozen now: a dev question is "time-type" if at least two of its disambiguated questions differ by a 4-digit year or an ordinal word. Reported descriptively; no decision depends on it.
2. If S2 or S4 fails, the topology claim is withdrawn for this setting, whatever P1 shows.

Exclusions

- **Train/dev overlap:** a dev question whose text matches a train question exactly (after lowercasing and stripping punctuation) is excluded. Near-duplicates (token Jaccard ≥ 0.8) are counted, and P1 is re-reported without them.

- Questions with no annotation.
- Questions where retrieval returns fewer than k chunks.
- Questions whose neighbourhood leaves fewer than 4 chunks after the topic join.

All are counted and reported **per label**, since an exclusion that correlates with the label would bias the result.

What each outcome means

- **P1 win, with S2 and S4 passing:** the scoreboard and the utility study become the headline. Claim 2 gets its own pre-registration and an API cost estimate first.
- **Anything else:** published as it is. Topology is framed as a diagnostic, and the winning retrieval-only predictor is recommended for production.
- External context, not an endpoint: LLM-sampling methods report AUROC ≈ 0.67 on AmbigQA (a 2026 preprint, unverified). It may be shown next to our numbers, labelled with its different cost class.
- Examples shown in any illustration are a seeded random sample of dev, including failures.

Compute

- The dump download, page extraction and embeddings run on separate idle compute machines at the lowest scheduling priority, and dump size is checked first.
- Features, KDE and bootstrap run locally after sizing on 100 train questions; anything sized above ~10 min goes to a separate compute machine.
- No API calls.

Deviations after the lock (append-only; each dated)

1. **2026-10-15 19:10: dev label aggregate seen before Stage 2.** The corpus-sizing probe on `ambignq.zip` (downloaded 19:08) printed dev's ambiguity prevalence under the registered label. That breaks the rule that no dev label is computed before the Stage-2 freeze. Reading dev's `nq_doc_title` was planned (dev pages are always in the corpus); the prevalence was not. No dev feature or retrieval had been computed, so no feature–label relation was seen, and no decision rule depends on prevalence. From now on, dev is read only for titles until the freeze.
   - Observed: **58.5%** of dev ambiguous (printed rounded to 3 decimals; not recounted, to avoid reading dev labels again) under "≥ 1 annotator says `multipleQAs`", vs the 51% this document quotes from Min et al. (the cause is unverified; their figure may use a different aggregation). The power statement at 58.5% is unchanged to two decimals: SE(Δ) 0.0126, half-width 0.025, power 0.89 at $\Delta = 0.04$ and 0.35 at 0.02.
   - Train: 9,887 questions have one annotation, 83 have two, 66 have three. "One generator per question" holds for 98.5%; the registered label applies as written to all.
2. **2026-10-15 19:25: topic features computed on full pages (the authors' decision).** The library clusters topics per document (its topic-clustering module; topic label = page + cluster). On a local test with random 384-d unit vectors (one draw per size), it **crashes on 5-chunk pages** (its HDBSCAN wrapper takes the argmax of an empty list). On 10, 20 and 60 chunks it finds 4, 6 and 21 "topics" in pure noise (outliers excluded), with mean silhouette 0.31–0.44. The locked corpus (≤ 5 chunks per page) therefore cannot produce the 3 topic features (`top_k_topic_spread`, `silhouette_score_mean`, `silhouette_score_std`) as registered.
   - Change: each gold page is chunked in full with the library's sliding-window token chunker (250 o200k tokens, its default; the paper's C250), embedded, and clustered with the library's topic clusterer. Only the first ≤ 5 chunks per page enter the retrieval corpus (cap unchanged). Pages the library cannot cluster count as one topic (label 0, silhouette 0); their number is reported.
   - Unaffected: topology, scale and doc-spread features, S2, S6, and every decision rule. Embedding beyond 50k chunks is for clustering only, on a separate compute machine.
   - The noise finding (topics found in pure noise) goes into the limitations section, marked as a single-draw local test.
   - Seeding: the library's UMAP has no `random_state`, so "seeded" is implemented as `np.random.seed(0)` before each page, with numba single-threaded. On real pages the library fails on some pages of ≤ 6 chunks (probe on 12 pages, 19:54); those take the fallback.

3. **2026-10-16 09:05: report-only check of the KDE's numerics (the authors' decision, 2026-10-17).** While re-implementing the feature maths independently, the fixture tests found that sklearn 1.3.2's tree-based `KernelDensity.score_samples` (used by the library and by `p_correct` here) returns wrong log densities for samples far from every training row: off by 114 and 340 nats on two 300-row fixtures, flipping `p_correct` (e.g. 1.0 vs an exact 1.5e-5). The same KDE with `leaf_size=1000` (a single leaf, so brute force) returns the exact values. Nearer the data the two agree to 3e-15.
   - Change: the analysis script also computes `p_correct` from the exact Gaussian sum over the KDE's training rows and reports, for the primary cell: the share of dev questions where the two differ by $> 1\text{e-}6$, the largest difference, dev AUROC both ways (the method, topology-only, no-topology), and P1's Δ with the exact score.
   - Unchanged: every endpoint and decision rule still uses the library's numbers. Re-running all endpoints on the saved mock table (2026-10-17 12:30) reproduced every locked mock result exactly; on the mock the two scores agree to 6e-14 on all dev questions.
4. **2026-10-18 19:22: two report-only rivals, question length and an LLM judge (the authors' decision, 2026-10-18).** A red-team review of an illustrative example (2026-10-17) found that word count alone separates its 10 hand-picked illustrative queries, and that the cheapest rival in production is to ask the answering LLM itself. No dev retrieval or feature has been computed and Stage 2 is not frozen; the only dev label aggregate seen is deviation 1's prevalence.
   - **Question length:** the number of `\w+` tokens in the lowercased question. Its sign is fixed on val, as for every unsupervised predictor.
   - **LLM judge:** a small commercial LLM served through an API (model name withheld in this anonymised copy), temperature 0, at most 8 output tokens, one call per dev question. It sees the question and the top-5 retrieved chunks (MiniLM, the primary embedder), which is what a RAG answer call would see. Score = the integer it returns; a reply that is not an integer in 0–100 scores 50, and those are counted. Prompt, frozen now (`{passages}` is the 5 chunk texts, each preceded by its page title, separated by blank lines):

     > Here are passages retrieved for a question.\n\n{passages}\n\nQuestion: {question}\n\nCould this question reasonably have more than one correct answer, because it can be read in more than one way? Reply with only an integer from 0 (certainly one answer) to 100 (certainly several).

   - **Report-only.** Neither rival can become "the baseline" for P1, and no endpoint or decision rule changes. For each, on both labels: dev AUROC with its 95% CI, and the paired ΔAUROC (the method − rival) with the registered bootstrap.
   - **Caveats, stated now.** AmbigQA has been public since 2020, so the judge may have seen its questions or labels in training; its AUROC is an upper bound for unseen questions. It is a different cost class (one API call per query), reported as such.
   - **This amends "Compute: No API calls"** for the judge only: about 2,002 calls. The cost is estimated before running, the calling script is hashed with the Stage 2 freeze, and it runs only with the authors' API key and approval. If it is not run, it is reported as not run. The claim's own predictors stay LLM-free.
5. **2026-10-19 09:02: four report-only checks on what the topology features measure (the authors' decision, 2026-10-19).** Toy models built for exposition found that `w1_h0`, the mean of the 49 H0 bars, tracks how spread out each meaning is as much as whether a second meaning exists. In a 384-d model (two meanings far apart, 60 samples per row, one seed each, 2026-10-19 08:53), its AUROC for one meaning vs two was 1.00 with equal spread, and fell to 0.96 / 0.83 / 0.62 / 0.53 (spread in 2 / 5 / 20 / 383 directions) when spread varied $\times 0.5$–1.5 between samples; the longest H0 bar's stayed at 1.00. Neither number separated the 10 hand-picked illustrative queries, whose meanings overlap. On those queries the top eigenvalue holds 0.12–0.29 of the offsets' variance, against 0.036 for a matched-cosine null (2026-10-18). As in deviation 4: no dev retrieval or feature has been computed, Stage 2 is not frozen, and the only dev label aggregate seen is deviation 1's prevalence.
   - **Longest H0 bar:** the largest finite H0 lifetime, in the library's convention (so on `w1_h0`'s scale). Reported alone and beside `w1_h0` alone, each a single predictor with its sign fixed on val: dev AUROC with 95% CI, and their paired ΔAUROC with the registered bootstrap.
   - **Top-eigenvalue share:** the 50 unit offsets `(c_i - q) / ‖c_i - q‖`, centred; the largest eigenvalue of their Gram matrix over its trace. A single predictor, sign fixed on val, reported like deviation 4's

rivals.

- **Matched-cosine null:** each neighbour is replaced by a random direction at its own cosine to the query, `c_i' = (c_i·q) q + √(1 − (c_i·q)²) v_i` with `v_i` uniform and orthogonal to `q` (seed 0, one draw per question). The topology-only model is refitted and evaluated on these clouds, all else unchanged. Reported: its dev AUROC beside the real one, with the paired Δ. A topology signal that survives on the null comes from the cosines, which the retrieval-score predictors already see.
- **Per-query stability:** for 500 fit-split questions (seed 0), `w1_h0`, `lt_max_h1` and the longest bar are recomputed on 20 random subsets of 40 of the 50 neighbours. Reported: each feature's reliability, 1 − (mean within-question variance) / (variance over all subset values). No labels are used.
- **Report-only and exploratory:** no endpoint or decision rule changes, none of these can become P1's baseline, and no multiplicity correction is applied. Sized on 100 train questions before any run; anything over ~10 min goes to a separate compute machine.

6. **2026-10-30 09:15: what the test can conclude, an incremental endpoint, and external registration (the authors' decision, 2026-10-30).** An internal review of the paper judged that P1 cannot settle the question as written. P1 is powered at 0.35 for its own margin of 0.02, so a null result is uninformative. It also pits a nine-feature model against the best single predictor, while the paper's real-text work shows that the relevant rival is a flexible non-topological classifier. As in deviations 4 and 5: no dev retrieval or feature has been computed, Stage 2 is not frozen, and the only dev label aggregate seen is deviation 1's prevalence.
   - **New co-primary P1b (incremental value).** The rival is B8 (GBM, the library's class) on the retrieval-only predictors, plus the 50 sorted cosines and summaries of the 50 × 50 similarity matrix: the top 5 eigenvalues; the mean, median, 10th and 90th percentiles of the pairwise cosines; and the same percentiles of the nearest-neighbour cosines. The test model is the same GBM with `w1_h0`, `lt_max_h1` and the longest H0 bar added. Both are fitted on fit, with signs and hyperparameters set on val, and evaluated on dev with the registered paired bootstrap.
     - "Win": ΔAUROC ≥ 0.01 and the 95% CI excludes 0.
     - "No practically relevant gain": the whole 95% CI lies within ±0.02. This is an equivalence statement, reachable with SE ≈ 0.013.
     - Otherwise: "not demonstrated".
   - **P1 is unchanged and still reported.** Its power statement gains one sentence: P1 alone cannot show the absence of an effect; P1b's equivalence rule can.
   - **Report-only additions.** None changes a decision rule, and each is sized before any run.
     - (a) The corpus-bearing control: each neighbour keeps its cosine, and its bearing is taken from a random chunk of the search corpus outside the query's top 50. There are 20 draws, and their AUROCs are averaged. Reported: the real AUROC minus the control's, for `w1_h0`, `lt_max_h1` and the longest bar.
     - (b) `vmf2`, a two-meaning von Mises–Fisher statistic, as a single predictor with its sign fixed on val. Its definition is fixed in the authors' dated plan of 2026-10-30, 09:00: the 50 unit offsets projected onto their top 5 principal directions, then BIC(one component) − BIC(two components).
     - (c) Corpus sensitivity: P1 and P1b repeated with the corpus enlarged by 50,000 chunks from random non-gold pages of the same dump (seed 0).
   - **External registration.** This plan, with this deviation, is registered on OSF (embargoed) before Stage 2. The Stage-2 freeze record adds the registration identifier.
   - **Not adopted:**
     - A second benchmark with clarification-need labels: a different task and label, noted as future work.
     - Further K sensitivity: {25, 50, 100} is already registered.

Freeze record

Stage 1: status line set to LOCKED (2026-10-15 19:07) · Stage 2 analysis-script hash: (pending) · dev run timestamp: (pending)

v1 → v2 changes (red team 2026-10-15 15:05; re-run on v2 at 15:20)

1. The corpus no longer uses `viewed_doc_titles` (label leakage). It uses the AmbigQA 2020-01-20 dump at a fixed date.
2. Added QPP predictors (NQC, coherence, Clarity, conditionally Dense-QPP) and direction spread R to the comparison set.
3. Added the isotropic-direction null (S4) and topology vs direction spread (S2).
4. The baseline is chosen on val, and the CI is a single 95% interval.
5. Added the power statement and redefined the sensitivity-flip rule.
6. Pinned the pushed library commit, the dump date and seeds; two-stage lock.
7. Registered the ambiguity-type prediction, with a frozen stratifier.
8. From the v2 re-run: dropped train questions leave the fit set; NQC and Clarity are named as adaptations, with tokenisation fixed; S4 is relabelled as a sanity floor; the prediction is reworded to match its stratifier; exclusions are reported per label.

v2 → v3 changes (dry run on synthetic data, 2026-10-15 16:20)

1. B6 belongs to the retrieval-only comparison set; v2 wrongly listed it under "fitted on the same features".
2. Unsupervised predictors get their sign on val. B4 and B5 are reported as one, since they rank identically.
3. **Train/dev overlap check added.** In the mock, train and dev shared random streams, and GBM "beat" the 20% label-noise ceiling (0.91 vs ≤ 0.74) by memorising duplicates. NQ-open has a known risk of the same kind.
4. KDE balancing is seeded (library bug fixed in a pinned commit). Latency is reported per method, because B6 alone costs ~25 ms per question, more than the method's features.

v3 → v4 changes (dry-run decisions by the authors, 2026-10-15 16:45)

1. A sign flip counts only when $|\Delta| \geq 0.01$. In the mock, cells at $\Delta \approx +0.001$ voided a primary of -0.037.
2. The arm reports the fraction of neighbours kept and is labelled vacuous at $\geq 95\%$. In the mock, $= 1.4$ moved no cell by more than 0.002.
3. **S6 added: topology-only KDE vs the baseline.** In the mock, the 9-feature KDE (0.616) ranked below its topology subset (0.694) and below logistic/GBM on the same features (~0.70). Feature spreads differ ~130-fold, and one bandwidth lets the widest feature dominate; standardising helped only partly (0.629). The primary stays the library's method.

v4 → v5 changes (2026-10-15 18:35)

1. **Dense-QPP is in**, as the adaptation defined under Methods (the authors, 18:30). The paper and its code disagree on three points: variance vs std, 30 samples vs one vector, RBO vs average overlap. Each is resolved explicitly and stated.
2. Red team on v5 (internal review, 18:55): (s, K) moved from val to fit tuning, because a 90-way choice on val biases the baseline pick in the method's favour.
3. The data interface gains a `search` callable for the depth-1,000 lists. The mock searches a per-question pool of 1,000 chunks, a smoke test of the code path only.